Architecture du moteur Magic

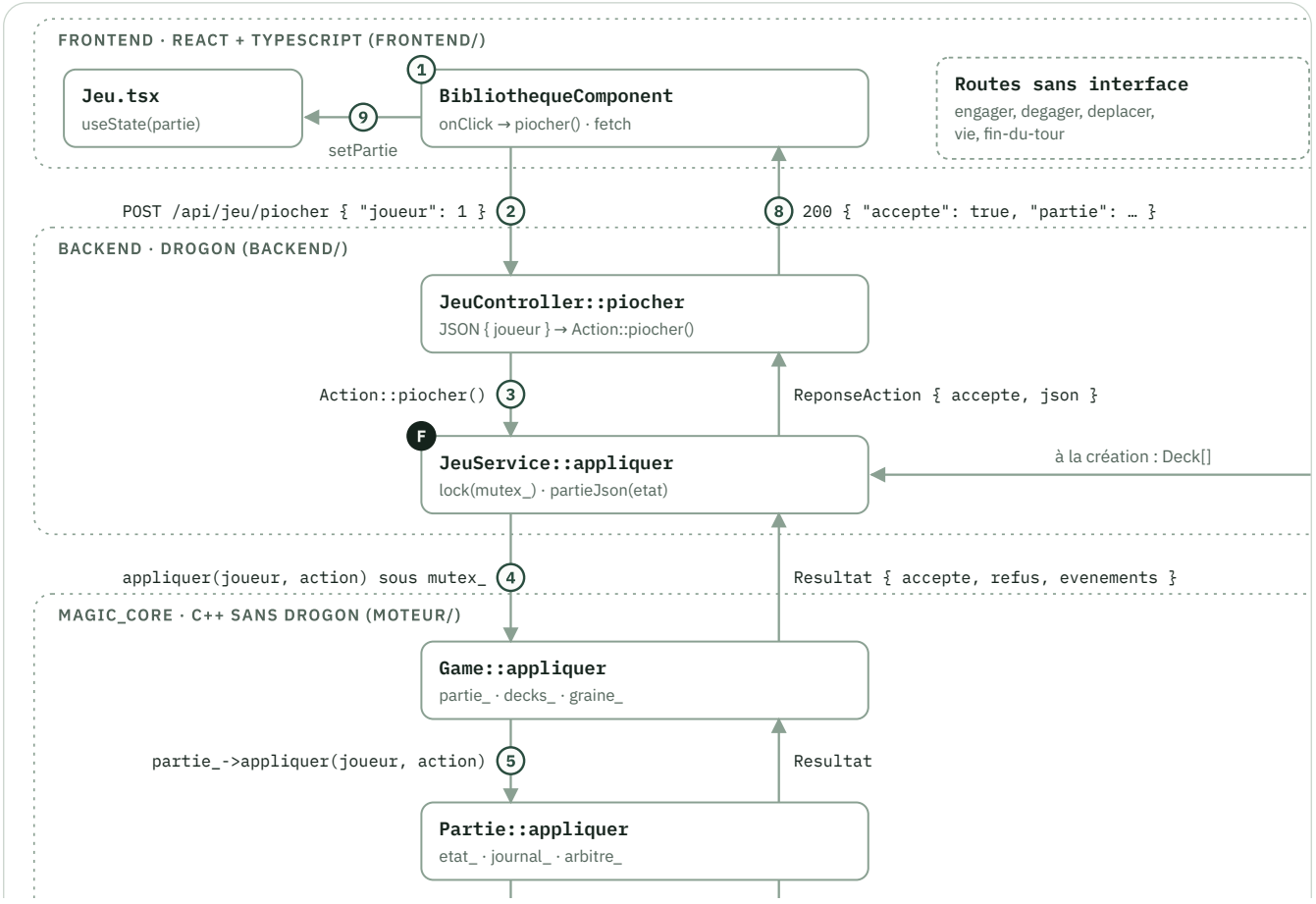
Le moteur tel qu'il est écrit aujourd'hui, puis la structure visée pour jouer une partie complète : tours, combat, pile, cartes de Kaladesh et explications des règles.

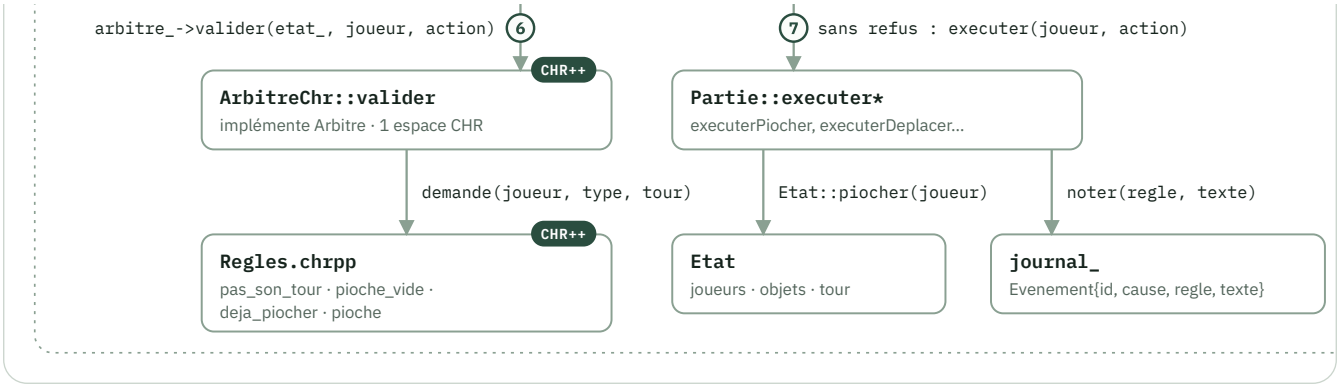
Existe À étendre À créer `CHR++` Règles écrites en CHR++

Aujourd'hui Structure cible Ajouter une règle Feuille de route À trancher

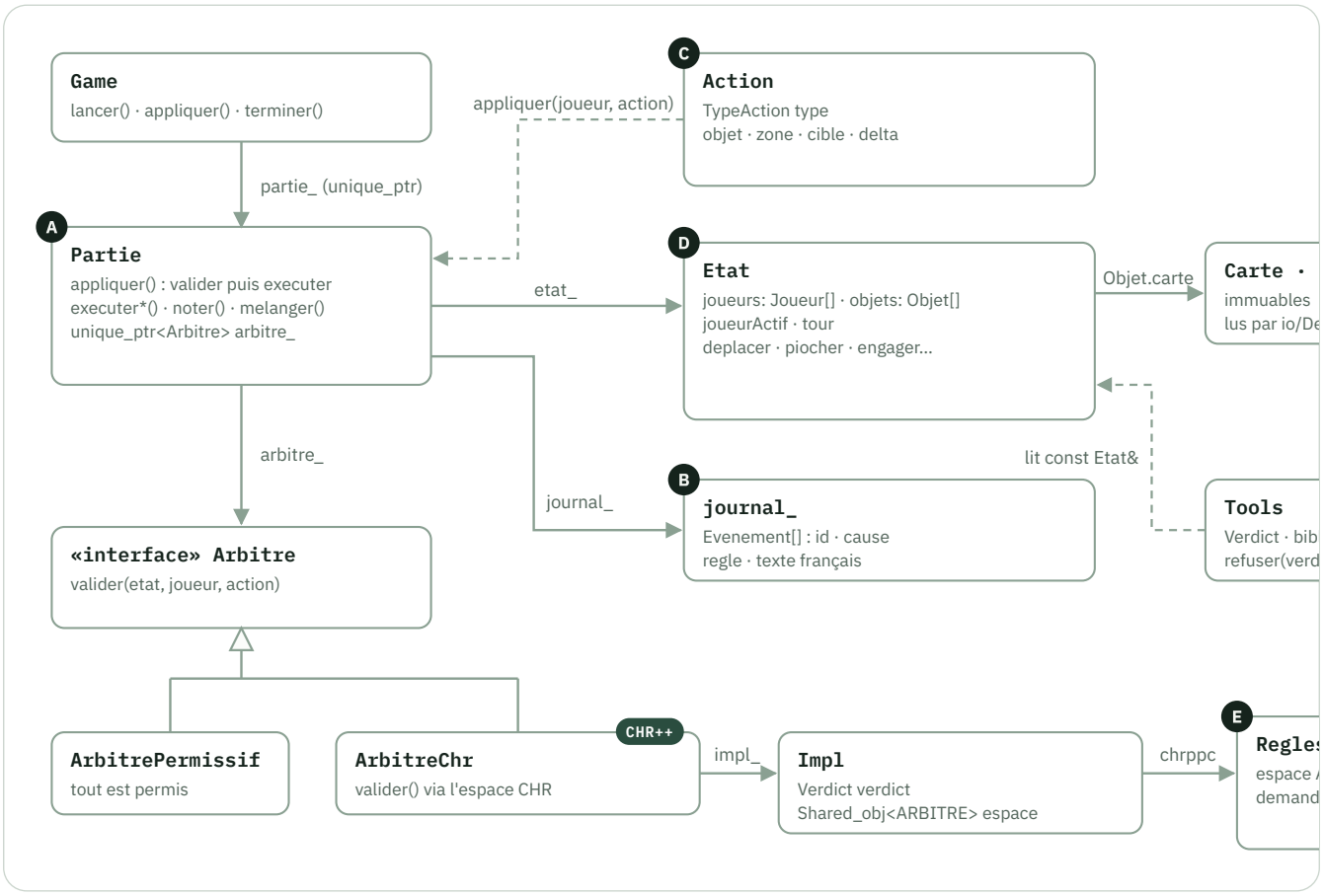
Aujourd'hui

Une action de jeu traverse le frontend, Drogon puis `magic_core`. Le moteur ne connaît ni Drogon ni le HTTP : il reçoit une `Action` et rend un `Resultat`. Quatre règles CHR++ valident les actions : c'est le bon tour, la bibliothèque n'est pas vide, une seule pioche par tour, et la pioche acceptée est enregistrée. Le schéma de `docs/` (frontend, backend C++, SQLite) reste exact ; celui-ci détaille la boîte du moteur.





Chemin d'une pioche, du clic à l'affichage. Les cinq autres routes existent côté backend, mais aucun composant ne les appelle encore.



Ce que contient `magic_core`. `Partie` possède l'état, le journal et l'arbitre ; seul `ArbitreChr` connaît CHR++. Les lettres renvoient à la liste ci-dessous.

Ce qui gêne déjà

A `Partie.cpp` contient tous les `executer*`. Chaque nouvelle action y ajoute une fonction et un `case`.

B Un `Evenement` porte déjà la phrase française, construite dans `executer*`. La règle et son texte ne sont pas séparés, donc pas d'explicateur à gabarits.

C `Action` est une struct plate : `objet`, `zone`, `cible` et `delta` n'ont de sens que selon

le `type`. Il n'y a pas de place pour une liste d'attaquants.

D `Etat` n'a ni phase, ni pile, ni combat, ni joueur attendu. `pas_son_tour` ne sait comparer qu'à `joueurActif`.

E La mémoire `a_pioche` vit dans le store CHR, alors que le reste du jeu vit dans `Etat`. Deux endroits à tenir d'accord.

F `partieJson` envoie les deux mains en clair. C'est correct tant que les deux joueurs partagent un écran.

Structure cible

Trois règles de conception, pour que le moteur reste simple à modifier quand les règles se multiplient.

Etat est la seule vérité

Des données et des opérations de base, sans règle. Les règles CHR++ le lisent dans leurs gardes ; le store CHR ne garde que ce qui se décide maintenant.

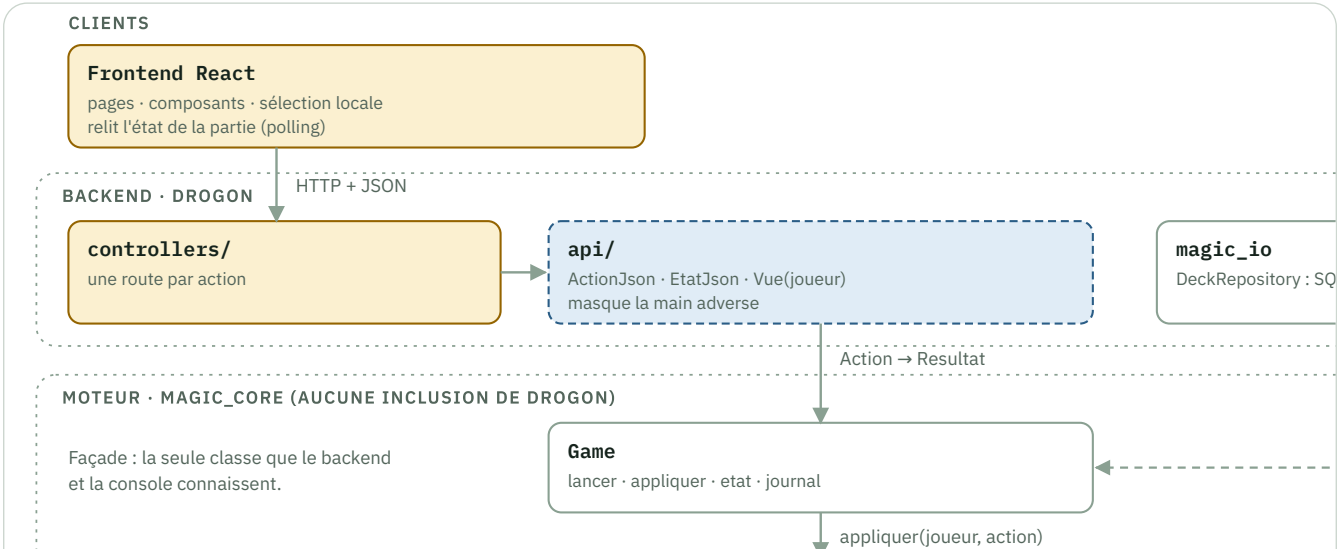
Chaque action suit le même chemin

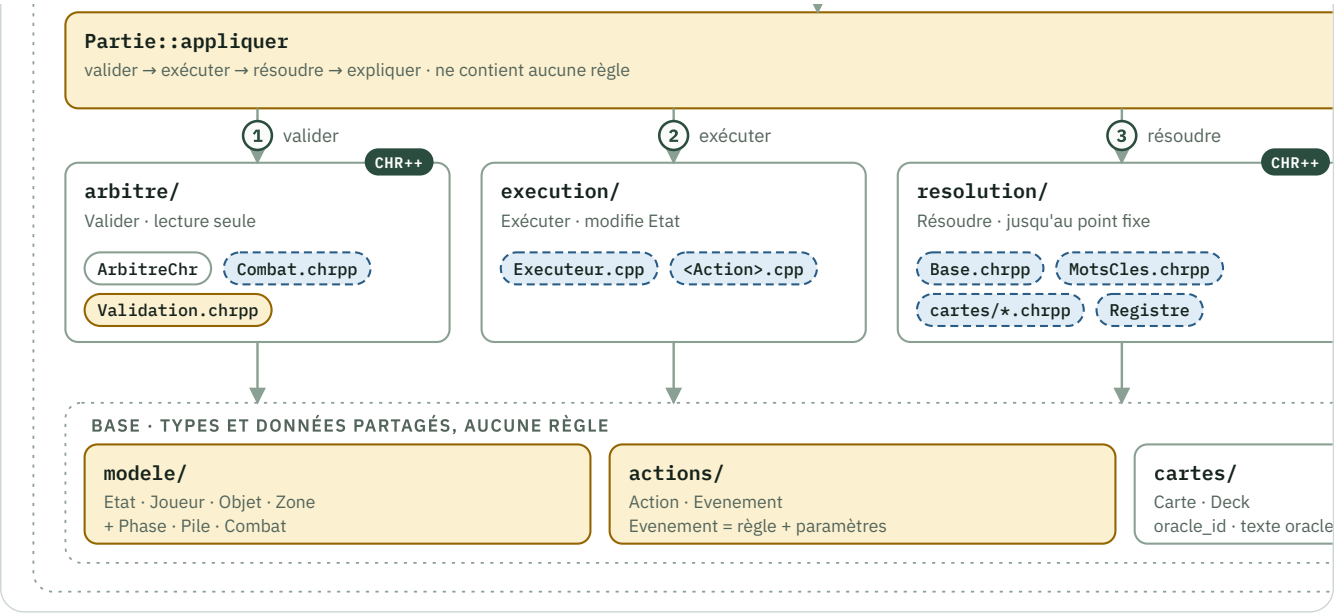
Valider, exécuter, résoudre, expliquer. `Partie` orchestre ces quatre étapes et ne connaît aucune règle.

Ajouter une règle, c'est ajouter un fichier

Un nouveau type d'action coûte une valeur d'enum et un `case`. Le reste vit dans de nouveaux fichiers.

Existe À étendre À créer `CHR++` Règles écrites en CHR++

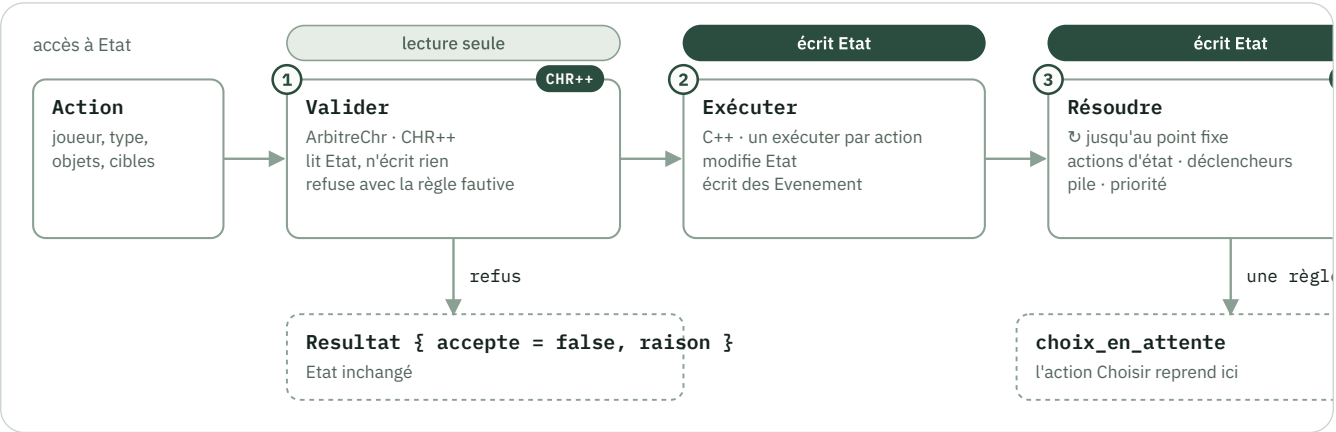




Chaque couche ne dépend que de celles du dessous. Les chiffres 1 à 4 sont l'ordre des étapes d'une action, détaillé juste après. Le trait pointillé est la dépendance directe de la console sur `Game`.

Le chemin d'une action

Toutes les actions passent par les mêmes quatre étapes, dans le même ordre. Seule la validation lit l'état sans l'écrire : un refus ne laisse aucune trace.

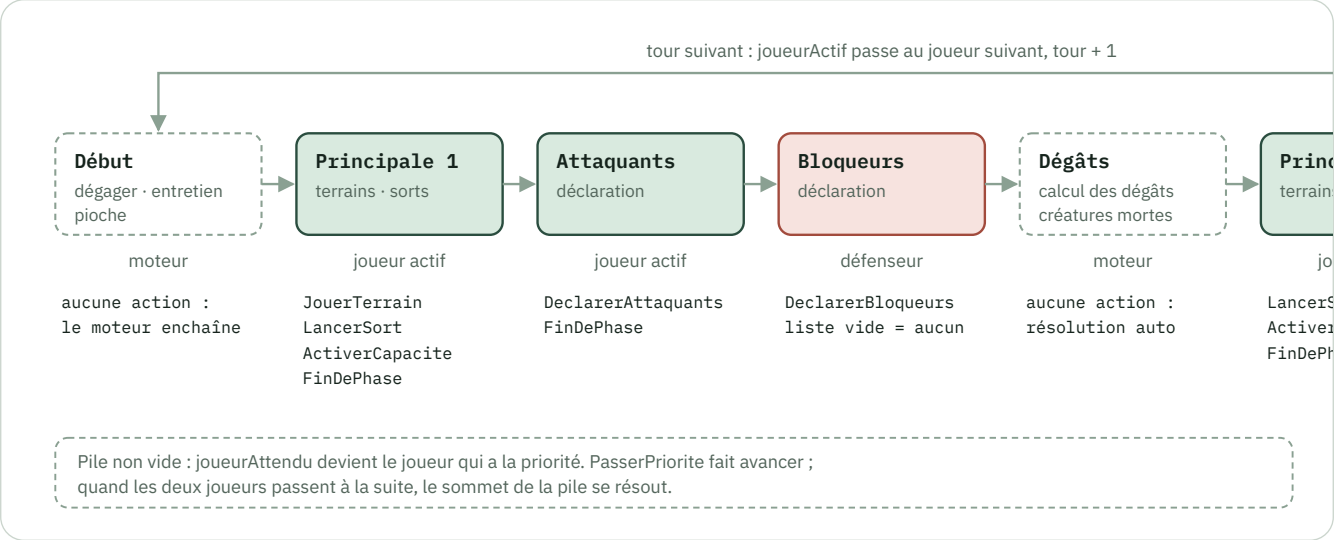


La barre du haut montre où l'état peut changer. Une règle qui demande un choix au joueur arrête la résolution, et l'action suivante la reprend.

> **Contrainte de chrppc**, vérifiée avec la version 1.90 : il lit un seul fichier par appel, et un espace CHR (un bloc `<CHR name="…">`) ne se répartit pas sur plusieurs fichiers. Un fichier peut contenir plusieurs espaces, chacun avec son propre store. Les règles se rangent donc par famille (validation, combat, base, cartes) et non par action. Ce qu'une règle doit retenir d'un coup à l'autre vit dans `Etat`.

Le tour de jeu

Aujourd'hui `Etat` ne connaît que `joueurActif` et `tour`. Deux champs suffisent à piloter les phases, le combat et la pile : `phase` et `joueurAttendu`.



La couleur d'un état dit qui peut agir. La pioche du début de tour devient automatique : la règle `deja_piocher` disparaît et l'action `Piocher` reste pour les effets de cartes.

Ce qui change dans les données

TYPE	AUJOURD'HUI	CIBLE
`Etat`	`joueurs`, `objets`, `joueurActif`, `tour`	+ `phase`, `joueurAttendu`, `pile`, `combat`
`Objet`	`id`, `carte`, `proprietaire`, `zone`, `engage`	+ `controleur`, `degats`, `arriveCeTour`, `marqueurs`
`Joueur`	`nom`, `vie`, `zones`	+ `mana`, `terrainsJoues`, `energie`
`Action`	struct plate : `type`, `objet`, `zone`, `cible`, `delta`	même struct, + `objets` (liste d' `ObjetId`) et `blocages` (liste de `{bloqueur, attaquant}`)
`Evenement`	`id`, `cause`, `regle`, `texte`	`id`, `cause`, `regle`, `parametres` ; le texte vient de l'explicateur

Tout reste en `struct`, `enum` et `vector` : pas de `variant` ni d' `optional`.

Ajouter une règle : où je touche

Exemple : déclarer les attaquants. La cible vise zéro fichier modifié dans `Partie`.

COUCHE	AUJOURD'HUI	CIBLE
Type d'action	`Action.hpp` : `TypeAction` et `Action::declarerAttaquants()`	Identique : une valeur d'enum, une fabrique
Validation	`Regles.chrpp` : nouvelles règles dans l'espace `ARBITRE`	Règles dans l'espace de la famille (`Validation.chrpp`, `Combat.chrpp`)
Message de refus	`ArbitreChr.cpp` : un `if` de plus dans `translate()`	Un gabarit dans `explication/`, sans code
Exécution	`Partie.hpp` et `Partie.cpp` : `executerX()`, un `case`, la phrase dans `noter()`	`execution/DeclarerAttaquants.cpp` et un `case` dans `Executeur.cpp`
Backend	`JeuController` : route et lecture JSON ; `JeuService` : JSON de sortie	Une route et une fonction dans `ActionJson`
Console	`Console.cpp` : une commande	Identique
Fichiers de `Partie` touchés	**2**	**0**

Feuille de route

Chaque étape s'appuie sur la précédente. La première ne change aucun comportement : elle prépare le terrain.

1 Ranger avant d'ajouter

Sortir les `executer*` de `Partie.cpp` vers un `Executeur`, faire porter à `Evenement` la règle et ses paramètres plutôt que la phrase, créer `moteur/test/` avec un scénario par règle. Les étapes suivantes ajoutent toutes des actions : ce découpage coûte moins cher maintenant.

2 Le tour

`Phase`, `joueurAttendu`, début de tour automatique (dégager, pioche). `pas_son_tour` compare à `joueurAttendu`.

3 Terrains et mana

Type de carte lu dans `ligneType`, `JouerTerrain` (un par tour), réserve de mana, `FinDePhase`.

4 Combat

`DeclarerAttaquants`, `DeclarerBloqueurs`, dégâts, créatures mortes. C'est le premier cas où le joueur attendu change en plein tour.

5 Explications

Explicateur et gabarits français par règle, test « aucune règle muette », journal exposé en JSON. Le sujet du Master porte sur ce point : il doit exister avant que les règles se multiplient.

6 Pile et priorité

`LancerSort` (coût, cibles), `PasserPriorite`, résolution de la pile, `choix_en_attente` et `Choisir`.

7 Cartes de Kaladesh

Registre par `oracle_id` (« non implémentée » sinon), mots-clés partagés, énergie, véhicules, fabrication.

8 Plus tard

Plusieurs parties à la fois (`GestionnaireParties`) et `Vue(joueur)` pour masquer la main adverse quand chacun jouera sur son écran.

À trancher

Où vit ce qu'une règle retient d'un coup à l'autre ?

Recommandation : dans Etat. Un fait gardé dans un store CHR n'est visible que de son espace, et les règles seront réparties sur plusieurs espaces. `Etat` se copie pour un retour arrière et se sérialise. Aujourd'hui `a_pioche` suit l'autre voie : elle marche tant qu'il n'y a qu'un espace. Avec la mémoire dans `Etat`, l'espace créé une seule fois n'est plus nécessaire et peut redevenir recréé à chaque validation, sans `demande` périmée.

Règles de carte : un .chrpp par carte ou des données ?

Recommandation : des .chrpp, comme le prévoit `structure.md`, avec les mots-clés

partagés dans leur propre espace. Des briques génériques (piocher N, créer un jeton) ne se justifient que si les cartes simples deviennent nombreuses.

Qui a la priorité dans la première version ?

Recommandation : le second joueur humain sur le même écran, comme aujourd'hui. `joueurAttendu` couvre aussi l'adversaire passif ou une IA : seul l'appelant change.

Plusieurs parties en même temps ?

Recommandation : pas avant l'étape 8. `JeuService` avec un seul `Game` suffit tant que les deux joueurs partagent un écran.

Établi à partir de `moteur/`, `backend/`, `frontend/` et `structure.md`, le 22 octobre 2026. Le comportement de chrppc a été testé sur la version 1.90 installée.